

AGENTIC AI MASTERY

6-Month Complete Roadmap

From Computer Engineering Student → Top-Tier Company

Python • LLMs • LangChain • CrewAI • Vector DBs • Multi-Agent Systems

Structured | Week-by-Week | Project-Driven | Placement-Ready

*Crafted exclusively for ambitious Computer Engineering students
aiming for Tier-1 placements in AI/ML roles · 2025 Edition*

■ TABLE OF CONTENTS

01	Introduction & How to Use This Roadmap	pg 3
02	Stage 1 — Foundations (Weeks 1–6)	pg 4
03	Stage 2 — Chatbots vs Agents + Architecture (Weeks 7–12)	pg 6
04	Stage 3 — Agent Frameworks + Memory (Weeks 13–18)	pg 8
05	Stage 4 — Tools, Multi-Agent, Production & Deployment (Weeks 19–24)	pg 10
06	Complete Week-by-Week Schedule	pg 12
07	Resource Library (Free & Beginner-Friendly)	pg 16
08	Projects — Mini + Major (Placement-Level)	pg 19
09	Skill Stack Summary	pg 22
10	Daily/Weekly Study System & Consistency Tips	pg 23
11	Placement Strategy — Resume, Interviews, Companies	pg 24

01 — Introduction & How to Use This Roadmap

Who this is for: Computer Engineering students with programming knowledge (Java, basic web dev) who want to master Agentic AI from scratch and land a top-tier role in 6 months or less.

What is Agentic AI?

Agentic AI refers to AI systems that can **plan, decide, use tools, and take autonomous actions** to achieve goals — going far beyond simple chatbots. Think of an AI that can browse the web, write and run code, send emails, and coordinate other AI agents — all to complete a task you gave it in plain English. This is the frontier of AI engineering and the fastest-growing job category in tech.

Why This Roadmap Puts You Ahead of 90% of Students

- Most students learn Python or ML theory — you'll learn to BUILD production-grade AI agents
- Portfolio projects are designed to impress at Google, Microsoft, OpenAI, Anthropic, and Indian unicorns
- Every topic is directly mapped to what hiring managers test in 2025 interviews
- Buffer weeks and a burnout-prevention system are baked in
- Resources are 100% free and beginner-friendly — no paid courses required

How to Use This Document

- **Follow stages in order.** Each stage builds on the previous. Don't skip ahead.
- **Use the week-by-week schedule.** Treat it as your GPS, not a strict law. Adjust if needed.
- **Build every project.** Projects are non-negotiable. They ARE your resume.
- **Track with a simple checklist.** Each week, check off topics and projects completed.
- **Return to the resources section** whenever you feel lost on a topic.

Why this stage matters: You cannot build agents without a solid Python foundation, understanding of APIs, and a practical grasp of how LLMs work. This stage converts your Java/web background into AI-ready skills in 6 focused weeks.

Python for AI (Weeks 1–2)

- Python syntax crash course — variables, loops, functions, OOP
- File I/O: reading/writing JSON, CSV, TXT (critical for agents that store state)
- List comprehensions, lambda, map/filter — write concise AI code
- Exception handling — agents must fail gracefully
- Virtual environments (venv, conda) — professional project setup
- pip package management — install and manage AI libraries
- Type hints & docstrings — needed when contributing to AI repos
- Async Python (asyncio, await) — agents often run multiple tasks concurrently

APIs & HTTP (Week 3)

- What is a REST API? Understanding endpoints, verbs (GET/POST/PUT/DELETE)
- Using the `requests` library — fetching data from any web API
- Authentication: API keys, Bearer tokens, OAuth basics
- Reading API documentation (OpenAI, Anthropic, weather APIs, GitHub API)
- Rate limiting, retries, and error handling in API calls
- Building a simple FastAPI server — agents need to expose endpoints too
- JSON parsing and schema validation with Pydantic
- Environment variables for secrets — never hardcode API keys

Basic ML Concepts (Week 4)

- What is Machine Learning? Supervised vs Unsupervised vs Reinforcement
- Training, Validation, Test splits — understanding overfitting
- Key algorithms (conceptual): Linear Regression, Decision Trees, Random Forests
- Embeddings — what they are and WHY they power every modern AI agent
- Cosine similarity — how agents find 'similar' memories or documents
- NumPy basics: arrays, matrix ops (required for LLM I/O manipulation)
- Pandas basics: DataFrames, reading CSVs, cleaning data
- Matplotlib/Seaborn: visualizing AI outputs and metrics

LLMs — How They Work (Week 5)

- What is a Large Language Model? Transformers in plain English
- Tokens, context windows, temperature, top-p — parameters every AI engineer tweaks
- Prompt engineering fundamentals: system prompts, few-shot, chain-of-thought
- OpenAI API: chat completions, function calling, streaming responses

- Anthropic Claude API: messages API, tool use, vision inputs
- Hugging Face — loading open-source models (Llama, Mistral) locally
- Comparing models: GPT-4, Claude 3.5, Gemini, Llama 3 — when to use which
- Cost management: tokens vs cost, batching, caching strategies

Putting It Together (Week 6)

- Build a prompt engineering playground — test prompts programmatically
- Structured outputs: forcing LLMs to return JSON using Pydantic + function calling
- Streaming responses for better UX
- Simple RAG prototype: load a PDF → chunk → embed → query
- Revision + buffer for anything missed in Weeks 1–5

Expected Outcomes After Stage 1

- Write clean Python for AI tasks confidently
- Call any REST API and handle responses/errors
- Understand how LLMs work and how to prompt them effectively
- Build a basic RAG (retrieval-augmented generation) prototype
- Set up professional Python project structure

Why this stage matters: Most people build chatbots and call them 'AI agents'. Understanding the architectural difference is what separates junior from senior AI engineers. This stage gives you the conceptual and practical depth that impresses interviewers.

Chatbots vs Agents — Deep Dive (Week 7)

- Chatbots: stateless Q&A, fixed rules, no real decision-making
- Agents: goal-directed, tool-using, iterative, self-correcting systems
- The ReAct pattern: Reasoning + Acting in a loop (foundation of all agents)
- The cognitive architecture of an agent: Perceive → Plan → Act → Observe → Reflect
- When to use a chatbot vs an agent (business decision, not just technical)
- LLM as the 'brain': how the model decides what action to take next
- Tool calling / function calling — how agents interact with the world
- Error recovery: what happens when an agent's action fails

Retrieval-Augmented Generation (RAG) — Full System (Weeks 8–9)

- Why RAG? Solving the knowledge cutoff and hallucination problems
- Document loading: PDF, web pages, Notion, CSV — using LangChain loaders
- Text splitting strategies: recursive, semantic, token-aware chunking
- Embedding models: OpenAI text-embedding-3, Hugging Face sentence transformers
- Vector stores: Chroma (local), Pinecone (cloud), Weaviate, FAISS
- Similarity search vs MMR (Maximal Marginal Relevance) retrieval
- Hybrid search: combining vector + keyword (BM25) search
- RAG evaluation: context precision, answer faithfulness with RAGAS
- Advanced RAG patterns: HyDE, parent-document retrieval, re-ranking

Memory Systems for Agents (Week 10)

- 4 types of agent memory: sensory (current context), short-term (conversation), long-term (vector DB), episodic (past experiences)
- Conversation buffer memory vs summary memory — tradeoffs
- Persistent memory: saving/loading agent state to disk or DB
- Semantic memory: storing facts as embeddings, retrieving by relevance
- Entity memory: tracking people, places, and concepts across conversations
- Building a memory module from scratch (not just using a framework blindly)
- When memory hurts performance: token budget management

Agent Reasoning Patterns (Week 11)

- Chain-of-Thought (CoT): forcing step-by-step reasoning
- Tree-of-Thought (ToT): exploring multiple reasoning paths

- ReAct: interleaving reasoning and tool use
- Reflexion: agents that learn from their failures in the same session
- Plan-and-Execute: pre-plan all steps then execute (more deterministic)
- Self-ask with search: agents that generate and answer their own sub-questions
- Structured output parsing: handling malformed LLM responses robustly

Practical Agent Patterns + Stage Review (Week 12)

- Building a complete research agent from scratch (no framework)
- Implementing tool routing: how an agent decides which tool to call
- Graceful degradation: what to do when tools fail or LLM times out
- Observability basics: logging agent steps, tokens, latency
- Stage 2 revision and portfolio documentation

Expected Outcomes After Stage 2

- Explain agent vs chatbot architecture clearly in interviews
- Build a full RAG pipeline from document to answer
- Implement 4 types of agent memory
- Code a ReAct agent from scratch without frameworks
- Evaluate and debug agent reasoning failures

Why this stage matters: Frameworks are what professional AI engineers use daily. LangChain, LangGraph, and CrewAI appear in almost every AI job posting. After Stage 2's conceptual foundation, frameworks will make sense — not be black boxes.

LangChain Deep Dive (Weeks 13–14)

- LangChain architecture: chains, runnables, LCEL (LangChain Expression Language)
- LLMChain → modern Runnable pattern migration (many tutorials are outdated!)
- LangChain tools: built-in (search, calculator, Python REPL) + custom tools
- Agents in LangChain: AgentExecutor, create_react_agent, create_tool_calling_agent
- LangChain memory integrations: ConversationBufferMemory, VectorStoreRetrieverMemory
- LangChain document loaders: 50+ supported formats
- Output parsers: PydanticOutputParser, StructuredOutputParser
- LangSmith for tracing, debugging, and evaluating your chains
- LangServe: deploying LangChain apps as REST APIs

LangGraph — Stateful Agent Workflows (Week 15)

- Why LangGraph? Handling complex, cyclical agent workflows
- Graph concepts: nodes, edges, state, conditional routing
- Building a stateful agent: defining state schema with TypedDict
- Conditional edges: routing agent flow based on tool output
- Human-in-the-loop: pausing graph execution for human approval
- Subgraphs: composing complex agents from smaller agent modules
- Streaming agent steps to the frontend in real-time
- LangGraph vs LangChain: when to use each

CrewAI — Multi-Agent Collaboration (Week 16)

- CrewAI concepts: Agents, Tasks, Tools, Crews
- Defining agent roles, goals, and backstories for better performance
- Sequential vs hierarchical crew processes
- CrewAI tools: SerperDev, web scraping, file read/write, custom tools
- Inter-agent communication: how agents pass context to each other
- Crew output: structured task results and final deliverables
- Real example: a research crew (Researcher + Analyst + Writer agents)

Microsoft AutoGen (Week 17)

- AutoGen ConversableAgent: the universal multi-agent interface
- UserProxyAgent: the human-in-the-loop agent
- AssistantAgent: the LLM-powered worker agent
- GroupChat: orchestrating N agents in a round-table discussion

- Code execution in AutoGen: let agents write and run their own code
- AutoGen nested chats: agents spawning sub-conversations
- AutoGen vs CrewAI: different philosophies, when each shines

Advanced Memory + Stage Review (Week 18)

- Building a persistent agent memory layer with a vector DB
- Connecting Redis for fast short-term agent memory
- PostgreSQL + pgvector for production-grade memory
- Memory consolidation: summarizing old memories to save context
- Comparative study: LangChain vs LangGraph vs CrewAI vs AutoGen
- Stage 3 project completion and portfolio prep
- Buffer week: catch up or explore one framework deeper

Expected Outcomes After Stage 3

- Build production-grade agents with LangChain/LangGraph
- Coordinate multiple specialized agents with CrewAI
- Implement persistent memory with vector + relational databases
- Trace, debug, and evaluate agent pipelines with LangSmith
- Choose the right framework for any agent architecture

Why this stage matters: Building agents locally is one thing — shipping them is another. This stage covers the full production lifecycle: tool ecosystems, MCP servers, Docker, cloud deployment, monitoring, and security. This is what separates portfolio projects from real products.

Tool Ecosystems & MCP (Week 19)

- What is MCP (Model Context Protocol)? Anthropic's standard for tool-agent communication
- Building custom MCP servers: exposing your own tools to any AI agent
- Tool categories: search, code execution, database, file system, web browsing, APIs
- Brave Search API / Serper / Tavily — choosing the right search tool
- Browser-use / Playwright for web automation by agents
- E2B Code Interpreter — secure sandboxed code execution for agents
- Computer-use APIs: agents that control desktops
- Tool reliability patterns: retries, timeouts, fallback tools

Multi-Agent System Architecture (Week 20)

- Orchestrator-Worker pattern: one agent directing many specialist agents
- Peer-to-peer agent networks: agents that negotiate and vote
- Event-driven agents: reacting to database events, webhook triggers
- Message queues for agents: Redis Pub/Sub, Kafka basics
- Shared state management in multi-agent systems
- Conflict resolution: what happens when agents disagree
- Designing agent personas for reliability (role + goal + constraint)
- Load balancing across agents: when tasks are too big for one agent

Databases for AI (Week 21)

- Pinecone: production vector database, namespaces, metadata filtering
- Weaviate: hybrid search, generative search capabilities
- Qdrant: high-performance vector DB, on-premise deployment
- pgvector (PostgreSQL extension): vector search in a relational DB
- Redis Vector Search: ultra-fast in-memory vector operations
- Choosing a vector DB: cost, scale, latency, features comparison
- Relational DBs for agent state: SQLite (dev), PostgreSQL (prod)
- MongoDB for unstructured agent logs and conversation history

Deployment & Infrastructure (Week 22)

- Docker fundamentals: writing Dockerfiles for AI applications
- Docker Compose: running multi-service agent stacks locally
- FastAPI: building REST APIs around your agents

- Streamlit: rapid UI for agent demos (important for showcasing projects)
- Deploying to cloud: Railway, Render, Fly.io (free tiers for portfolios)
- AWS Lambda for serverless agent triggers (cost-efficient)
- Environment management: secrets with .env, AWS Secrets Manager
- GitHub Actions: CI/CD pipeline for your AI projects

Monitoring, Safety & Ethics (Week 23)

- LangSmith: production tracing, prompt management, evaluations
- Langfuse: open-source alternative for agent observability
- Key metrics: latency, cost per run, success rate, hallucination rate
- Guardrails AI / NeMo Guardrails: preventing harmful agent outputs
- Prompt injection attacks: how bad actors hijack agents and defenses
- Rate limiting and abuse prevention for agent APIs
- Data privacy: handling PII in agent pipelines, GDPR basics
- Responsible AI: bias in agent decisions, transparency requirements

Final Projects + Career Prep (Week 24)

- Complete Major Project 2 polish and documentation
- Write technical blog post about one of your projects (publish on Medium/Dev.to)
- GitHub portfolio cleanup: README templates, demo GIFs, architecture diagrams
- Prepare 'system design' answers for agent projects
- Mock interview on agent architecture with a peer
- LinkedIn profile update with project links
- Final revision of entire roadmap — consolidate all learnings

Expected Outcomes After Stage 4

- Deploy a multi-agent system to cloud with monitoring
- Build and expose custom tools via MCP
- Secure agent pipelines against prompt injection
- Set up observability dashboards for production agents
- Have 2 major projects fully polished for interviews

06 — Complete Week-by-Week Schedule

■ Recommended: 2–3 hours on weekdays · 4–5 hours on weekends · ~20 hrs/week total. Buffer weeks are included. If you miss days, don't skip — compress.

STAGE 1 — Foundations (Weeks 1–6)

Week	Study Topics	Practice	Build
Wk 1	Python crash course: syntax, OOP, files, exceptions, venv setup	Solve 10 LeetCode Easy problems in Python; rewrite 3 Java programs in Python	CLI tool: JSON to CSV converter
Wk 2	Advanced Python: async/await, type hints, list comprehensions, decorators	Build 3 async scripts that fetch data from public APIs concurrently	Async news fetcher: fetch headlines from 5 sites simultaneously
Wk 3	REST APIs: requests library, FastAPI, Pydantic, auth, rate limiting	Integrate 3 different public APIs (weather, GitHub, a quote API)	FastAPI micro-server with 3 endpoints + Pydantic validation
Wk 4	Basic ML: embeddings, cosine similarity, NumPy, Pandas, Matplotlib	Compute embeddings for 100 sentences; build a basic semantic search	Semantic search engine for your notes using sentence-transformers
Wk 5	LLMs: OpenAI/Anthropic API, tokens, prompting, function calling, streaming	Run 50 different prompts; compare outputs at different temperatures	Prompt engineering playground with side-by-side comparison UI
Wk 6	RAG prototype + Revision: chunk docs, embed, store in Chroma, query	Build RAG over a 20-page PDF and test 20 questions	Mini Project 1: Personal Document Q&A; bot (see Projects section)

STAGE 2 — Chatbots vs Agents + Architecture (Weeks 7–12)

Week	Study Topics	Practice	Build
Wk 7	Agent vs chatbot architecture; ReAct pattern; tool calling; cognitive loops	Re-implement a chatbot as a ReAct agent manually; trace reasoning steps	Simple ReAct agent that answers questions using web search (no framework)
Wk 8	RAG deep dive: loaders, chunking strategies, embedding models, vector stores	Compare 3 chunking strategies on the same document; measure retrieval quality	Advanced RAG: PDF + web page + CSV as knowledge base with hybrid search
Wk 9	Advanced RAG: HyDE, re-ranking, RAGAS evaluation, MMR retrieval	Evaluate your RAG system with RAGAS; improve score by 20%	Add evaluation dashboard to your RAG system using RAGAS metrics
Wk 10	Memory systems: 4 types; conversation buffer; summary; entity; vector memory	Build each type of memory and test with a 20-turn conversation	Agent with all 4 memory types; demo that it remembers facts from session 1 in session 2

Week	Study Topics	Practice	Build
Wk 11	Reasoning: CoT, ToT, Reflexion, Plan-and-Execute; output parsers	Implement CoT and compare output quality vs baseline; test Reflexion	Reflexion-based problem-solving agent that improves across 3 attempts
Wk 12	Build complete research agent from scratch + BUFFER / REVISION WEEK	Document your Stage 2 projects with architecture diagrams on GitHub	Research agent: given a topic, autonomously finds info + writes a summary report

STAGE 3 — Agent Frameworks + Memory (Weeks 13–18)

Week	Study Topics	Practice	Build
Wk 13	LangChain: LCEL, runnables, tools, AgentExecutor, LangSmith tracing	Rebuild your Stage 2 research agent using LangChain; compare code complexity	LangChain agent with 5 custom tools + LangSmith observability dashboard
Wk 14	LangChain advanced: memory integrations, output parsers, LangServe deployment	Deploy your LangChain agent as a REST API; test with Postman	LangServe-deployed chatbot API with persistent vector memory
Wk 15	LangGraph: nodes, edges, state, conditional routing, human-in-the-loop	Build 3 different LangGraph workflows; add human approval step	LangGraph approval workflow: agent that asks human before executing irreversible actions
Wk 16	CrewAI: agents, tasks, tools, sequential/hierarchical crews	Build a 3-agent research crew; observe inter-agent communication	Mini Project 3: CrewAI content creation crew (Researcher + Writer + Editor)
Wk 17	AutoGen: ConversableAgent, GroupChat, code execution, nested chats	Build a GroupChat with 4 agents debating a topic; add code-running agent	AutoGen coding team: agents that write, review, and test code collaboratively
Wk 18	Persistent memory (pgvector/Pinecone) + BUFFER / REVISION WEEK	Add persistent memory to your CrewAI project; test cross-session recall	Major Project 1 starts: AI Research Assistant (see Projects section)

STAGE 4 — Tools, Multi-Agent, Production & Deployment (Weeks 19–24)

Week	Study Topics	Practice	Build
Wk 19	MCP protocol, custom MCP servers, tool categories, Brave/Tavily, E2B sandbox	Build a custom MCP server exposing 3 tools; test with Claude Desktop	MCP server: expose a database query tool and a file-writing tool
Wk 20	Multi-agent architectures: orchestrator-worker, event-driven, message queues	Build orchestrator + 3 specialist workers; test with complex research task	Major Project 1 completion: deploy the full multi-agent research system
Wk 21	Vector DB deep dive: Pinecone, Qdrant, pgvector; hybrid search; metadata filters	Benchmark 3 vector DBs on the same dataset; measure latency and cost	Migrate your existing projects to Pinecone; add metadata filtering

Week	Study Topics	Practice	Build
Wk 22	Deployment: Docker, FastAPI, Streamlit, Railway/Render, GitHub Actions CI/CD	Dockerize 2 of your projects; set up CI/CD with GitHub Actions	Fully deployed agent app on Railway with Streamlit frontend + automated tests
Wk 23	Monitoring: LangSmith/Langfuse, guardrails, prompt injection defense, safety	Add observability to Major Project 2; test prompt injection attack and defend	Major Project 2 completion + monitoring dashboard (see Projects section)
Wk 24	FINAL WEEK: Portfolio polish, blog post, mock interviews, LinkedIn update	Publish a technical blog post; do 2 mock system design interviews	Complete GitHub portfolio: READMEs with demos, architecture diagrams, live links

07 — Resource Library (Free & Beginner-Friendly)

■ All resources listed here are FREE. Courses marked [MUST] are non-negotiable. Don't try to use all resources — pick one primary source per topic and go deep.

Python for AI

YouTube / Courses	Docs / Articles / Books
• [MUST] freeCodeCamp Python Full Course — youtube.com/watch?v=rfscVS0vtbw	• FastAPI Docs — fastapi.tiangolo.com (best documentation in Python ecosystem)
• Corey Schafer Python Tutorials Playlist — YouTube (best OOP + async coverage)	• Pydantic Docs — docs.pydantic.dev
• docs.python.org/3 — official docs, use as reference not tutorial	• Patrick Loeber ML Python Crash Course — YouTube
• Real Python (realpython.com) — deep articles on async, type hints, decorators	• Python Asyncio full guide — realpython.com/async-io-python
• Automate the Boring Stuff with Python — automatetheboringstuff.com (free book)	• Requests library docs — docs.python-requests.org

LLMs & Prompt Engineering

YouTube / Courses	Docs / Articles / Books
• [MUST] Andrej Karpathy 'Intro to LLMs' — YouTube (1hr, best conceptual overview)	• Simon Willison's blog — simonwillison.net (best practitioner blog on LLMs)
• [MUST] DeepLearning.AI 'ChatGPT Prompt Engineering for Developers' — free on deeplearning.ai	• Lilian Weng's blog (Li'Log) — lilianweng.github.io (research-depth articles)
• OpenAI API Documentation — platform.openai.com/docs	• HuggingFace Course — huggingface.co/learn (free, hands-on)
• Anthropic Claude API Docs — docs.anthropic.com	• LearnPrompting.org — free beginner-friendly prompt engineering course
• Prompt Engineering Guide — promptingguide.ai (comprehensive free resource)	• DeepLearning.AI 'Building Systems with ChatGPT API' — free short course

LangChain & LangGraph

YouTube / Courses	Docs / Articles / Books
• [MUST] LangChain Official Docs — python.langchain.com/docs (always most current)	• LangSmith Docs — docs.smith.langchain.com
• [MUST] LangGraph Docs — langchain-ai.github.io/langgraph	• LangChain GitHub repo — github.com/langchain-ai/langchain (read source code!)

• Alejandro AO LangChain Tutorials — YouTube (practical, project-based)	• DeepLearning.AI 'LangChain for LLM App Development' — free course
• LangChain YouTube channel — @LangChain (official tutorials)	• DataBrick's LLM course on edX — free audit available
• Greg Kamradt LangChain tutorials — YouTube	• LangChain blog — blog.langchain.dev

CrewAI & AutoGen

YouTube / Courses	Docs / Articles / Books
• [MUST] CrewAI Official Docs — docs.crewai.com	• DeepLearning.AI 'Multi AI Agent Systems with crewAI' — free course
• [MUST] AutoGen Documentation — microsoft.github.io/autogen	• DeepLearning.AI 'AI Agentic Design Patterns with AutoGen' — free course
• CrewAI YouTube channel — @CrewAI (official walkthroughs)	• Mervin Praisson CrewAI advanced tutorials — YouTube
• Brandon Hancock CrewAI tutorials — YouTube	• AssemblyAI Agent tutorials — YouTube
• AutoGen GitHub examples — github.com/microsoft/autogen/tree/main/samples	• The AI Advantage channel — YouTube (real-world agent builds)

Vector Databases & RAG

YouTube / Courses	Docs / Articles / Books
• [MUST] DeepLearning.AI 'Building and Evaluating Advanced RAG' — free course	• Weaviate Academy — weaviate.io/developers/academy (free)
• Pinecone Learning Center — pinecone.io/learn (free guides)	• pgvector GitHub — github.com/pgvector/pgvector
• Chroma Docs — docs.trychroma.com	• James Briggs RAG tutorials — YouTube (most practical RAG content)
• Qdrant Documentation — qdrant.tech/documentation	• Towards Data Science RAG articles — towardsdatascience.com
• RAGAS Documentation — docs.ragas.io	• Jerry Liu (LlamaIndex) talks on RAG — YouTube

Deployment & Production

YouTube / Courses	Docs / Articles / Books
• [MUST] Docker official 'Get Started' guide — docs.docker.com/get-started	• TechWorld with Nana Docker tutorial — YouTube (best Docker intro)
• FastAPI Production Deployment docs — fastapi.tiangolo.com/deployment	• Langfuse Docs — langfuse.com/docs
• Streamlit Docs — docs.streamlit.io	• Render Docs — render.com/docs
• Railway.app Docs — docs.railway.app	• Fly.io Docs — fly.io/docs

• [GitHub Actions Docs — docs.github.com/en/actions](https://docs.github.com/en/actions)

• [AWS Lambda Python tutorial — docs.aws.amazon.com](https://docs.aws.amazon.com/lambda/python/)

08 — Projects (Mini + Major) — Placement Level

■ Projects are your most powerful placement tool. Each project below is designed to be unique, technically impressive, and directly mappable to real business problems. Build them in public on GitHub.

■ MINI PROJECT 1 (End of Stage 1, Week 6)

Description	An intelligent Q&A system that ingests ANY document (PDF, DOCX, TXT, web URL) and lets you ask questions in natural language. Goes beyond simple RAG by maintaining conversation history and citing exact sources.
Features	Multi-format document loading • Semantic chunking with overlap • Chroma vector store • Conversation memory (last 10 turns) • Source citation with page numbers • Confidence scoring per answer • Export conversation as PDF
Tech Stack	Python · LangChain · OpenAI/Claude API · Chroma DB · sentence-transformers · Streamlit UI · PyPDF2 · BeautifulSoup
Resume Value	Shows RAG implementation, vector DB usage, API integration, and ability to ship a complete product with UI. Directly relevant to 'AI Engineer' and 'ML Engineer' JD requirements.
Agentic AI Link	Foundation for all future agent projects — this is your memory/knowledge module.

■ MINI PROJECT 2 (End of Stage 2, Week 12)

Description	A fully hand-coded ReAct agent (zero frameworks) that takes a research question, autonomously plans its research strategy, uses web search + Wikipedia + arXiv tools, synthesizes findings, and outputs a structured report.
Features	Manual ReAct loop implementation • 4 tools (web search, Wikipedia, arXiv API, calculator) • Chain-of-Thought reasoning trace • Reflexion module (retries on failure) • Structured markdown report output • Token usage tracking
Tech Stack	Python · OpenAI/Anthropic API · Tavily Search API · Wikipedia Python library · arXiv API · Pydantic · Rich (for CLI display)
Resume Value	Demonstrates deep understanding of agent internals — NOT just framework usage. Interviewers love this because most candidates can't explain how agents work under the hood.
Agentic AI Link	This IS the core of agentic AI: perception → reasoning → action → observation loop.

■ MINI PROJECT 3 (End of Stage 3, Week 16)

Description	A 4-agent content creation pipeline: a Trend Researcher finds viral topics, a Content Strategist plans the angle, a Writer drafts the article, and an Editor polishes and fact-checks. Runs autonomously from a single topic prompt.
Features	4 specialized CrewAI agents with distinct personas • Web search + RAG knowledge base tools • Hierarchical crew with orchestrator • Draft → Review → Publish pipeline • Output as formatted Markdown + HTML

Tech Stack	Python · CrewAI · LangChain · GPT-4 / Claude · Serper API · Chroma · FastAPI backend · Streamlit frontend
Resume Value	Multi-agent coordination is the hottest topic in AI engineering. This project directly shows competency in production-ready multi-agent systems.
Agentic AI Link	Demonstrates agent specialization, coordination, and task delegation — core multi-agent patterns.

■ MAJOR PROJECT 1 — AI-Powered Competitive Intelligence Platform

(Weeks 18–20 | Stage 3–4 Capstone)

Description	An end-to-end multi-agent system that continuously monitors competitors for a given company. Users input a company name; the system autonomously tracks news, product launches, job postings, pricing changes, and social sentiment — then generates weekly intelligence briefings.
Features	Orchestrator agent + 5 specialist agents (News, GitHub, LinkedIn, Pricing, Social) • LangGraph stateful workflow with scheduled triggers • Pinecone vector DB for historical intelligence storage • Cross-session memory: tracks changes over time • Automated weekly email briefing via SendGrid • Admin dashboard: Streamlit with real-time agent status • Export to PDF report • Comparison mode: track 3 competitors simultaneously
Tech Stack	Python · LangGraph · CrewAI · GPT-4 / Claude · Pinecone · PostgreSQL · Redis (caching) · FastAPI · Streamlit · Docker · Railway deployment · SendGrid · GitHub Actions CI/CD · LangSmith monitoring
Resume Value	This is a COMPLETE product solving a real B2B problem (companies pay \$500-5000/month for this). It demonstrates multi-agent orchestration, production deployment, scheduled workflows, monitoring, and business value. Will be the centerpiece of your portfolio.
How to Present	Frame it as: 'Built a production multi-agent system processing 500+ data points daily to generate competitive intelligence. Deployed on Railway with LangSmith monitoring, achieving <3s response latency.' Show the live demo and architecture diagram in interviews.
Agentic AI Link	Full stack: multi-agent coordination + persistent memory + scheduled execution + production deployment + observability.

■ MAJOR PROJECT 2 — Autonomous Software Development Agent

(Weeks 22–23 | Stage 4 Capstone)

Description	An autonomous coding agent that takes a feature request in plain English and independently: reads the codebase, writes a plan, implements the code, writes tests, runs them, debugs failures, and opens a pull request. Think: a junior developer that never sleeps.
Features	Codebase understanding via AST parsing + RAG on code files • LangGraph workflow: Plan → Implement → Test → Debug → PR loop • E2B sandbox for safe code execution • AutoGen GroupChat: Architect + Coder + Tester + Reviewer agents • GitHub integration: auto-create branches and PRs • Human-in-the-loop: pause for approval before PR • Supports Python + JavaScript codebases • Cost tracking per feature request • Web UI: input feature spec → watch agent work in real-time
Tech Stack	Python · LangGraph · AutoGen · GPT-4 / Claude 3.5 · E2B Code Interpreter · GitHub API (PyGithub) · AST module · Chroma (for code RAG) · FastAPI · Streamlit · Docker · Fly.io deployment · Langfuse monitoring

Resume Value	Code-generation agents are the #1 use case companies are building right now (Copilot, Devin, Cursor). This project puts you in the same conversation as those products. It demonstrates: agentic coding, tool use, code execution safety, human-in-the-loop design, and real software engineering integration.
How to Present	Demo it live: give it a simple feature request, show it write and test code in the sandbox. Say: 'Built an autonomous dev agent using LangGraph + AutoGen that reduced feature implementation time for small tasks by ~70% in testing.' Architecture diagram is key.
Agentic AI Link	The pinnacle of agentic AI: self-directed, self-correcting, tool-using agent on a complex real-world task.

09 — Complete Skill Stack Summary

Category	Skills / Tools	Proficiency Target
Languages	Python 3.11+, Bash scripting basics	Advanced
LLM APIs	OpenAI API, Anthropic Claude API, HuggingFace Inference API	Advanced
Prompt Engineering	CoT, Few-shot, System prompts, Function calling, Structured outputs	Advanced
Agent Frameworks	LangChain (LCEL), LangGraph, CrewAI, AutoGen	Intermediate-Advanced
RAG Systems	Document loaders, Chunking, Embeddings, Hybrid search, RAGAS evaluation	Advanced
Vector Databases	Chroma, Pinecone, Qdrant, pgvector, FAISS	Intermediate
Relational DBs	PostgreSQL, SQLite, Redis (caching/memory)	Intermediate
Memory Systems	Buffer, Summary, Entity, Vector, Persistent (cross-session)	Advanced
Tools / MCP	Tavily, Brave Search, E2B, Playwright, Custom MCP servers	Intermediate
Web / APIs	FastAPI, REST APIs, Pydantic, requests, async HTTP	Advanced
Frontend / UI	Streamlit, basic React (for demos)	Intermediate
DevOps / Deploy	Docker, Docker Compose, GitHub Actions CI/CD, Railway, Render, Fly.io	Intermediate
Observability	LangSmith, Langfuse, logging, cost tracking, latency metrics	Intermediate
Safety / Security	Prompt injection defense, Guardrails AI, secrets management	Intermediate
Data Tools	NumPy, Pandas, Matplotlib, JSON/CSV handling	Intermediate
Version Control	Git, GitHub (branches, PRs, issues, Actions)	Advanced

10 — Daily/Weekly Study System & Consistency Tips

Your Weekly Schedule Template

Day	Focus	Hours	Activity
Monday	New Theory	2 hrs	Watch 1 video + read docs on this week's topic
Tuesday	Hands-on Practice	2 hrs	Code along with tutorial; break things intentionally
Wednesday	Build	2.5 hrs	Work on the week's project (no watching, only building)
Thursday	Debug & Explore	2 hrs	Fix bugs, explore edge cases, read related GitHub repos
Friday	Light Review	1 hr	Review notes, update GitHub README, answer questions on forums
Saturday	Deep Build Session	4 hrs	Major project work + try a new feature you haven't done
Sunday	Consolidate	2 hrs	Write a short note on what you learned; plan next week

Consistency Rules (Non-Negotiable)

The 2-Minute Rule: If you miss a day, do SOMETHING for just 2 minutes — open VS Code, read one doc page. This breaks the inertia.

Never Miss Twice: Missing one day is an accident. Missing two days is building a habit. One miss = immediately plan makeup time.

Compress, Don't Skip: If you fall a week behind, compress: skip the nice-to-have subtopics, keep the core + project.

Weekly Anchor: Pick ONE specific time slot per week that is sacred (e.g., Saturday 10am-2pm). Protect it like an exam.

The 'Tiny Win' Habit: End every session by pushing SOMETHING to GitHub — even a small fix. Visual progress fights burnout.

Burnout Signal: If you feel dread before sessions 3 days in a row, take a 2-day complete break. Return with a fresh project.

Tracking System

Use a simple Notion or Google Sheet with 3 columns per week: ■ Done | ■ Partial | ■ Skipped. Review every Sunday. If you have 2+ ■ in a row, it's a signal to simplify the week's goal, not to quit.

11 — Placement Strategy: Resume, Interviews & Companies

How to Present Your Projects

The STAR format adapted for AI projects: **S**ituation (what problem), **T**ask (what the agent does), **A**rchitecture (how you built it — be specific about frameworks, vector DBs, agents), **R**esults (metrics: latency, accuracy, scale, cost reduction).

- Always have a LIVE DEMO or a recorded demo video (Loom/YouTube unlisted)
- Write an architecture diagram for every major project (draw.io or Excalidraw)
- GitHub READMEs must include: problem statement, architecture image, setup instructions, demo GIF
- Quantify everything: '87% retrieval accuracy on 500-question benchmark', '3-agent system processing 200 docs/hour'
- Write a Medium/Dev.to article for your major projects — shows communication skills
- Link your projects on LinkedIn BEFORE you start applying

Resume Tips for AI Roles

- Skills Section:** List exactly: Python, LangChain, LangGraph, CrewAI, OpenAI API, Anthropic API, Pinecone, ChromaDB, FastAPI, Docker, GitHub Actions
- Projects Section:** Use this format: [Project Name] — [1 sentence what it is] | Built with: [tech stack] | [2 bullet points with numbers]
- Avoid:** Generic terms like 'worked with AI' or 'used machine learning'. Be specific: 'implemented ReAct agent loop with LangGraph state management'
- ATS Keywords:** Include: LLM, RAG, vector database, agent, LangChain, multi-agent, embeddings, prompt engineering, FastAPI, Docker
- Length:** 1 page only. If you have too much, cut theory courses and keep projects + skills.
- GitHub link:** Make it prominent. Recruiters at AI companies look at GitHub before your degree.

What Interviewers Actually Test

Question Type	Example Question	How to Prepare
Conceptual	What is the difference between RAG and fine-tuning?	Be able to explain trade-offs: cost, freshness, accuracy, use cases
Architecture	Design a customer support agent for an e-commerce platform.	Practicing the 4Cs: Context, Components, Communication, Caveats
Code	Implement a basic ReAct loop using the OpenAI API.	Code it from scratch 3 times until it's muscle memory
Debugging	Your RAG returns irrelevant chunks. How do you debug?	Know chunking, embedding model choice, similarity threshold, reranking
Production	How would you handle 10,000 concurrent agent requests?	Asynchronous queues, worker pools, caching, horizontal scaling, rate limiting
ML Basics	What are embeddings? How do vector databases work?	Explain cosine similarity, approximate nearest neighbors (HNSW in Pinecone)

Target Companies & What They Look For

Company Tier	Companies	What They Look For
Tier 1 Global	Google DeepMind, Microsoft AI, Meta AI, OpenAI, Anthropic	Research depth OR strong engineering. Projects with novel architecture.
Tier 1 India	Flipkart, Meesho, Swiggy, CRED, Zepto, PhonePe (All of them)	Production deployment experience. Agent systems at scale. Python.
AI-First Startups	Sarvam AI, Krutrim, Yellow.ai, Haptik, Observe.AI	Build-fast mindset. Full-stack AI (frontend to agent to DB). Ship fast.
Big Tech India	Microsoft IDC, Google India, Amazon (AWS AI), Salesforce	CS fundamentals + AI application. System design. Open source.
Consulting	McKinsey QuantumBlack, BCG X, Deloitte AI	Problem framing, communication of AI solutions, business impact fr

30-Day Pre-Placement Sprint

- Week 1: Polish all GitHub repos — clean READMEs, live demo links, architecture diagrams
- Week 2: Write 2 technical blog posts, update LinkedIn, get 3 peer reviews of your projects
- Week 3: Do 5 mock system-design interviews focused on agent architecture
- Week 4: Apply aggressively — 5 applications/day. Use your blog posts as cold outreach openers

"The best time to start was 6 months ago. The second best time is today."
You now have everything you need. Execute the plan. Build the projects. Ship the products.

Good luck — you've got this. ■